嵌入式SDK接口规范

接口规范版本	更新内容	撰写者	时间
V1.0	嵌入式SDK接口规范第一版	王健康	2025-05-27
V1.1	1.增加sensor通道属性接口，包含翻转、镜像等操作 2.NTP时间接口中增加时间和日期格式字段	王健康	2025-06-12
V1.2	1.NTP设置时间增加不足1秒的余数，单位微秒 2.增加通用控制命令，用于启动对讲等控制命令 3.侦测事件接口增加声音告警控制	王健康	2025-06-18
V1.3	1.增加获取设备信息接口 2.增加CPE相关接口 3.device向host发消息，增加帧类型为：配网事件	王健康	2025-06-26
V1.4	1.增加host端状态通知 2.通用控制命令增加字段start_bind，用于配网之前，告诉IPC，提前准备好扫码程序等。 3.增加2.2.1配网信息说明，2.1中增加配网事件备注说明。	王健康	2025-07-02
V1.5	1.增加1.2.12 OTA升级接口	王健康	2025-07-08
V1.6	1.通用控制命令中增加白光灯控制 2.2.1节帧类型增加gps-rtk数据类型，类型为4	王健康	2025-07-28
V1.7	1.增加2.13获取SD卡信息接口 2.增加2.14获取网口信息接口 3.2.1节帧类型增加IPC复位类型，类型为5 4.通用控制命令中增加电子攻击的设置	王健康	2025-08-04
V1.8	1.2.2.2中增加gsensor数据，类型为6	王健康	2025-08-08
V1.9	1.2.2中增加音频帧，类型为7 2.1.2.12中ota升级增加ota_type、ota_status、start_result三个字段	王健康	2025-09-16

	3.增加2.16获取CPE固件版本接口		
V2.0	1.增加第4部分，UVC框架集成 2.增加第5部分，P2P框架集成	王健康	2025-09-18
V2.1	1.1.2.4中增加声音侦测 2.2.1中EventType增加声音侦测	王健康	2025-09-30
V2.2	1.增加1.2.17接口 2.增加1.2.18接口 3.增加1.2.19接口	王健康	2025-12-04
V2.3	1.增加6.关键定义，包括6.1和6.2	王健康	2025-12-17
V2.4	1.在1.2.6中增加字段：night_vision，用于设置夜视模式	王健康	2025-12-25
V2.5	1.在1.2.6中增加字段：start_gyro_data，用于产测验证gsensor数据获取	王健康	2025-12-26
V2.6	1.在1.2.6中night_vision字段，增加自动全彩夜视	王健康	2026-04-01
V2.7	1.在1.2.6中增加电子防抖控制开关，对应字段gyro_dis	王健康	2026-04-03

1.host下发命令到device

1.1实现方式说明

采用国科提供封装好的函数uvc_xu_ctrl用来设置和获取参数，接口和结构定义如下：

```
#define EXRNSION_UNIT_ID 0x0B
#define UVC_SET_CUR    0x01
#define UVC_GET_CUR    0x81
#define UVC_GET_MIN    0x82
#define UVC_GET_MAX    0x83
#define UVC_GET_RES    0x84
#define UVC_GET_LEN    0x85
#define UVC_GET_INFO   0x86
#define UVC_GET_DEF    0x87
struct uvc_xu_control_query {
```

```
    __u8 unit;
    __u8 selector;
    __u8 query;         /* Video Class-Specific Request Code, defined in linux/usb/video.h A.8.  */
    __u16 size;
    __u8 *data;
};
void uvc_xu_ctrl(unsigned int unit, unsigned int selector, unsigned int query, unsigned int
data_size, unsigned char* data_addr)
{
    int ret;
    struct uvc_xu_control_query x_ctrl = {0};
    x_ctrl.unit     = unit;
    x_ctrl.selector = selector;
    x_ctrl.query    = query;
    x_ctrl.size     = data_size;
    x_ctrl.data     = data_addr;
    ret = ioctl(g_fd, UVCIOC_CTRL_QUERY, &x_ctrl);
    if (ret < 0) {
        VIDEO_LOGE("UVCIOC_CTRL_QUERY fail");
    }
}
void uvc_xu_set(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char*
data_addr)
{
    // UVC_SET_CUR    0x01
    uvc_xu_ctrl(unit, selector, 0x01, data_size, data_addr);
}
void uvc_xu_get_cur(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned
char* data_addr)
{
    // UVC_GET_CUR    0x81
```

```
    uvc_xu_ctrl(unit, selector, 0x81, data_size, data_addr);
}
void uvc_xu_get_min(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char* data_addr)
{
    // UVC_GET_MIN    0x82
    uvc_xu_ctrl(unit, selector, 0x82, data_size, data_addr);
}
void uvc_xu_get_max(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char* data_addr)
{
    // UVC_GET_MAX    0x83
    uvc_xu_ctrl(unit, selector, 0x83, data_size, data_addr);
}
void uvc_xu_get_res(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char* data_addr)
{
    // UVC_GET_RES    0x84
    uvc_xu_ctrl(unit, selector, 0x84, data_size, data_addr);
}
void uvc_xu_get_len(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char* data_addr)
{
    // UVC_GET_LEN    0x85
    uvc_xu_ctrl(unit, selector, 0x85, data_size, data_addr);
}
void uvc_xu_get_info(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char* data_addr)
{
    // UVC_GET_INFO    0x86
  uvc_xu_ctrl(unit, selector, 0x86, data_size, data_addr);
}
```

```
void uvc_xu_get_def(unsigned int unit, unsigned int selector, unsigned int data_size, unsigned char* data_addr)
{
    // UVC_GET_DEF   0x87
    uvc_xu_ctrl(unit, selector, 0x87, data_size, data_addr);
}
```

数据结构struct uvc_xu_control_query的参数说明如下：

字段名称	类型	定义	备注
unit	u8	这个填EXRNSION_UNIT_ID，表示扩展命令	
selector	u8	命令ID	**国科目前只支持32个，所以每个接口要设计高效**
query	u8	对参数进行以下操作： 0x01，设置值 0x81，获取值 0x82，获取范围的最小值 0x83，获取范围的最大值 0x84，表示步长 0x85，获取参数值字节长度 0x86，表示是否支持获取和设置操作，0x1支持获取，0x2支持设置，0x3是都支持 0x87，获取默认值	
size	u16	参数字节长度	**这里支持60个字节**
data	u8*	参数指针	

1.2接口定义

1.2.1视频参数（命令ID：0x01）

字段名称	类型	定义	备注
mask	u32	操作类型	

		bit0：switch_on bit1：width&height bit2：venc bit3：osd bit4：time_location bit5：logo_location bit6：rate_ctrl_mode bit7：osd_reverse bit8：GOP bit9：frame_rate bit10：rate bit11：key_frame bit12：human_box	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
channel	u8	通道	通道定义： Chan = i*N+j i表示第i路sensor，范围0~2 N是每个sensor最多通道个数，目前是3 j是第i路sensor中第j路通道，范围0~2 sensor0: 0 1 2 sensor1: 3 4 5 sensor2: 6 7 8 sensor3: 9 10 11
switch_on	u8	通道开关 0：关闭通道 1：打开通道	对于uvc device，只要uvc host下发打开通道，uvc device一直向uvc host发送视频流，直到uvc host下发关闭通道。对于下发关闭，uvc device需要销毁通道，避免占用资源。
width	u16	宽度	
height	u16	高度	
venc	u8	编码格式 1：H264 2：H265	

		3：YUV 4：MJPEG	
osd	u8	是否开启osd 0x0：关闭时间和logo 0x1：打开时间 0x2：打开logo 0x3：打开时间和logo	
time_location	u32	时间水印坐标 bit0~bit15：x bit16~bit31：y	注意YUV不考虑OSD
logo_location	u32	logo水印坐标 bit0~bit15：x bit16~bit31：y	
rate_ctrl_mode	u8	码率控制模式 1：CBR 2：VBR 3：CQP	
osd_reverse	u8	是否反色 0：不反色 1：反色	
GOP	u8	I帧间隔	
frame_rate	u8	帧率	
rate	u16	码率，单位是K	
key_frame	u8	强制I帧 1：强制I帧 其他：忽略	
human_box	u8	是否开启人形框 0：不开启 1：开启	
res[32]	u8	预留扩展	注意确保4字节对齐

1.2.2NTP时间（命令ID：0x02）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：time_zone bit1：time bit2：osd_time_format bit3：osd_date_format	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
time_zone	s32	时区	+/-XXYY +：东时区 -：西时区 XX：小时 YY：分钟 比如：830表示东八区30分钟，-930表示西九区30分钟
time_sec	u32	时间戳	1970年以来的时间，单位秒，注意设置时间之前必须先设置时区
time_usec	u32	不足1秒的余数，单位微妙	1970年以来的时间，不足1秒的余数，单位微妙
osd_time_format	u8	OSD时间格式 0：24h 1：12h	
osd_date_format	u8	OSD日期格式 0：MM/dd/yyyy 1：yyyy/MM/dd	
res[42]	u8	预留扩展	注意确保4字节对齐

1.2.3音频参数（命令ID：0x03）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：switch_on	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合

		bit1：aenc bit2：bits_per_sample bit3：sample_rate bit4：volume bit5：mute	操作，可以单独设置某些bit位，这样可以提高接口使用效率。
channel	u8	音频通道 bit7：0表示麦克 bit7：1表示喇叭	目前只支持1路音频采集，默认是0
switch_on	u8	通道开关 0：关闭 1：开启	对于uvc device，只要uvc host下发打开通道，uvc device一直向uvc host发送音频流，直到uvc host下发关闭通道。
aenc	u8	音频编码格式 g711a：1 aac：2 pcm：3	这个接口不需要支持，由盒子端作编码处理
bits_per_sample	u8	每个采样位宽	这个接口暂不支持，陈林与潘远遗沟通过
sample_rate	u16	采样率	这个接口暂不支持，陈林与潘远遗沟通过
volume	u8	音量，1~100	如果是麦克通道，则是麦克音量 如果是喇叭通道，则是喇叭音量
mute	u8	是否静音 0：不静音 1：要静音	如果是麦克通道，则是控制麦克静音 如果是喇叭通道，则是控制喇叭静音
res[48]	u8	预留扩展	注意确保4字节对齐

1.2.4侦测事件（命令ID：0x04）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：region_point_1~region_point_6	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。

		bit1：motion_on bit2：motion_sensitivity bit3：human_on bit4：human_sensitivity bit5：face_on bit6：face_sensitivity bit7：motion_audio_alarm bit8：human_audio_alarm bit9：face_audio_alarm bit10：sound_det_on bit11：sound_det_sensitivity	
region_point_1	u32	侦测区域第1个点 bit0~bit15：x bit16~bit31：y	目前侦测区域默认1个六边形，如果要增加多个，再增加命令ID
region_point_2	u32	侦测区域第2个点 bit0~bit15：x bit16~bit31：y	
region_point_3	u32	侦测区域第3个点 bit0~bit15：x bit16~bit31：y	
region_point_4	u32	侦测区域第4个点 bit0~bit15：x bit16~bit31：y	
region_point_5	u32	侦测区域第5个点 bit0~bit15：x bit16~bit31：y	
region_point_6	u32	侦测区域第6个点 bit0~bit15：x bit16~bit31：y	
motion_on	u8	0：关闭移动侦测 1：打开移动侦测	

motion_sensitivity	u8	移动侦测灵敏度，1~100	
human_on	u8	0：关闭人形侦测 1：打开人形侦测	
human_sensitivity	u8	人形侦测灵敏度，1~100	
face_on	u8	0：关闭人脸侦测 1：打开人脸侦测	
face_sensitivity	u8	人脸侦测灵敏度，1~100	
motion_audio_alarm	u8	0：关闭移动侦测播放声音告警 1：使能移动侦测播放声音告警	
human_audio_alarm	u8	0：关闭人形检测播放声音告警 1：使能人形检测播放声音告警	
face_audio_alarm	u8	0：关闭人脸检测播放声音告警 1：使能人脸检测播放声音告警	
sound_det_on	u8	0：关闭声音侦测 1：打开声音侦测	
sound_det_sensitivity	u8	声音侦测灵敏度： 50：50db 60：60db 70：70db 80：80db 90：90db	
res[21]	u8	预留扩展	注意确保4字节对齐

1.2.5sensor通道属性（命令ID：0x05）

字段名称	类型	定义	备注

mask	u32	操作类型 bit0：mirror bit1：flip	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
dev_id	u8	Sensor ID，表示第几路sensor	
mirror	u8	是否开启镜像 0：不开启 1：开启	
flip	u8	是否开启翻转 0：不开启 1：开启	
res[53]	u8	预留扩展	注意确保4字节对齐

1.2.6通用控制命令（命令ID：0x06）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：start_talk bit1：start_bind bit2：white_led_on bit3：ele_attack_1 bit4：ele_attack_2 bit5：ele_light_1 bit6：ele_light_2 bit7：night_vision bit8：start_gyro_data	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
start_talk	u8	是否启动对讲 0：停止对讲 1：启动对讲	
start_bind	u8	是否启动配网绑定 0：停止配网	这个用于配网之前，告诉IPC，提前准备好扫码程序等。

		1：启动配网	
white_led_on	u8	是否打开白光灯 0: 关闭白光灯 1: 打开白光灯	
ele_attack1	u8	电子枪1攻击 0：停止攻击 1：启动攻击	
ele_attack2	u8	电子枪2攻击 0：停止攻击 1：启动攻击	
ele_light1	u8	电子枪1照明 0：停止照明 1：启动照明	
ele_light2	u8	电子枪2照明 0：停止照明 1：启动照明	
night_vision	u8	设置夜视模式 1：黑白模式，白天画面为彩色，晚上设备开启红外灯，画面切黑白。 2：全彩夜视，不论环境光强弱，摄像机将始终保持彩色画面，晚上自动开启白光灯。 3：自动全彩夜视，白天不亮灯，夜晚默认不亮灯，只有出现人形，亮灯持续15秒。	当手动打开白光灯，夜视模式失效，即手动开启白光灯优先级最高。手动关闭白光灯后，恢复夜视模式。
start_gyro_data	u8	是否启动获取防抖数据 0：停止获取防抖数据 1：启动获取防抖数据	
gyro_dis	u8	电子防抖 0：关闭电子防抖 1：打开电子防抖	
res[46]	u8	预留扩展	注意确保4字节对齐

1.2.7获取设备信息（命令ID：0x07）

字段名称	类型	定义	备注
version[32]	u8	设备版本号	获取设备版本号
res[28]	u8	预留扩展	注意确保4字节对齐

1.2.8CPE控制（命令ID：0x08）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：cpe_mode	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
cpe_mode	u8	CPE模式 0：自动模式 1：4G模式 2：中继模式	1.如果是中继模式，必须同时设置set_ssid、encryption、password_len、ssid、password_tail
set_ssid	u8	是否下发WIFI的ssid 0：不下发，使用已下发过的 1：下发新的SSID	
encryption	u8	中继模式下，要连接的WIFI是否加密，如果是加密的，要使用password 0：不加密 1：加密	如果要下发新的SSID，并且是加密的，则要执行1.2.9中命令，下发WIFI密码
password_len	u8	WIFI密码长度	WIFI密码由1.2.9中命令下发，如果长度大于60，则password_tail填充剩余部分。WIFI标准中密码最大长度为63
ssid[32]	u8	WiFi SSID	WIFI标准中SSID最大长度为32
password_tail[4]	u8	WiFi密码尾部，超出60字节的部分	
res[16]	u8	预留扩展	注意确保4字节对齐

1.2.9下发WIFI密码（命令ID：0x09）

字段名称	类型	定义	备注
password[60]	u8	WIFI密码	如果长度大于60，则剩余部分由1.2.8中password_tail填充。WIFI标准中密码最大长度为63

1.2.10获取CPE信息（命令ID：0x0A）

字段名称	类型	定义	备注
percent	u32	接收信号的强度百分比	如果CPE连接的是4G，则返回4G信号强度。 如果CPE连接的是WIFI，则返回WIFI信号强度。
dbm	s32	接收信号的dbm值	如果CPE连接的是4G，则返回4G信号强度。 如果CPE连接的是WIFI，则返回WIFI信号强度。
cpe_mode	u8	CPE模式 0：自动模式 1：4G模式 2：中继模式	
res[51]	u8	预留扩展	注意确保4字节对齐

1.2.11host端状态通知（命令ID：0x0B）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：host_status	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
host_status	u32	host端状态 0：prepare_binding，进入待配网状态	

		1：connect_internet，NVDIA Orin 连接internet中 2：connect_cpe，NVDIA Orin 连上CPE 3：connect_cloud，NVDIA Orin 连上云服务器 4：other_net_error，配网异常-其它网络原因导致无法连接云服务器 5：bind_account，绑定云账号 6：disconnect_cloud，云平台断连 7：reconnect_cloud，云平台断连后又恢复重连	
res[52]	u8	预留扩展	注意确保4字节对齐

1.2.12OTA升级（命令ID：0x0C）

字段名称	类型	定义	备注
length	u32	升级文件长度	
md5[32]	u8	OTA文件的md5	
start	u8	启动传输 1：启动传输 0：结束传输	
ota_type	u8	OTA类型： 1：IPC固件升级 2：CPE固件升级	
ota_status	u8	获取当前升级状态 1：没有任何升级 2：正在执行升级	启动升级之前，可以先获取当前ota状态，确认当前是否有其他升级任务，如果返回0，确认没有其他升级任务，则可以启动升级。
start_result	u8	获取启动升级结果 1：启动成功 2：升级冲突，比如有其他升级 3：启动失败，其他原因 4：还没有启动	1）如果没有执行start动作，则start_result返回可以忽略。 2）如果收到还没有启动，则等一会再查询，直到启动成功。如果没有启动成功，

res[20]	u8	预留扩展	注意确保4字节对齐

说明：

1）对于IPC升级，升级过程中，会杀掉媒体进程，host端无法与IPC通信

2）对于CPE升级，升级过程中，媒体进程正常运行，host端可以与IPC正常通信

1.2.13获取SD卡信息（命令ID：0x0D）

字段名称	类型	定义	备注
total_size	s32	SD卡总大小	SD卡总容量，单位是M
used_size	s32	SD卡已使用大小	SD卡已使用大小，单位是M
status	u8	SD卡状态 0：拔掉状态 1：插入状态 2：插入新卡，没有分区 3：卡异常	
volume[11]	u8	SD卡卷标	
res[40]	u8	预留扩展	注意确保4字节对齐

1.2.14获取网口信息（命令ID：0x0E）

字段名称	类型	定义	备注
duplex	u8	模式 1：full，全双工 2：half，半双工 3：unknown，未知	
operstate	u8	操作状态 1：up 2：down	
speed	s16	网速，单位Mbps，比如100Mbps、1000Mbps	

res[56]	u8	预留扩展	注意确保4字节对齐

1.2.15设置host端信息（命令ID：0x0F）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0：MAC	mask默认值是0xFFFFFFFF，表示所有字段都有效。为了支持单个操作和几个组合操作，可以单独设置某些bit位，这样可以提高接口使用效率。
MAC[18]	u8	host端MAC地址	
res[38]	u8	预留扩展	注意确保4字节对齐

1.2.16获取CPE固件版本（命令ID：0x10）

字段名称	类型	定义	备注
version[32]	u8	CPE版本号	获取CPE版本号
res[28]	u8	预留扩展	注意确保4字节对齐

1.2.17设置产测信息（命令ID：0x11）

字段名称	类型	定义	备注
mask	u32	操作类型 bit0: factory_info	
factory_info_type	u8	产测信息类型： 1：SN 2：DID 3：MAC 4：PRODUCT	
factory_info[30]	u8	是否开启镜像 0：不开启	注意MAC要加上:

		1：开启	
res[25]	u8	预留扩展	

1.2.18获取产测DID和SN（命令ID：0x12）

字段名称	类型	定义	备注
DID[30]	u8	设备DID	
SN[30]	u8	设备SN	

1.2.19获取MAC信息（命令ID：0x13）

字段名称	类型	定义	备注
MAC[18]	u8	主SOC的MAC	注意MAC是加上:
product[18]	u8	产品型号	
res[24]	u8	预留扩展	

2.device向host发送消息

device向host发送消息，目前与国科确定的方案是通过uvc数据通道，发送到host端。

uvc采用usb bulk模式，传输更稳定，而uac采用iso模式，存在不稳定，传输数据要满足period_size，也不灵活。

2.1帧头定义

字段名称	类型	定义	备注
HeaderFlag	u32	头部标识	#define MAKEFLAG(ch0, ch1, ch2, ch3) \ ((unsigned int)(char)(ch0) | ((unsigned int)(char)(ch1) << 8) | \ ((unsigned int)(char)(ch2) << 16) | ((unsigned int)(char)(ch3) << 24))

			#define UVC_FRAME_HEADER_FLAG MAKEFLAG('D','Y','6','6')
DataLen	u32	数据长度，包括帧头和帧数据	
FrameType	u8	帧类型 1：视频 2：侦测事件 3：配网事件 4：gps-rtk数据 5：IPC复位事件 6：gsensor数据 7：音频	
KeyFrame	u8	0：非关键帧 1：关键帧	只针对视频帧
Channel	u8	通道	对于视频，表示某个通道 对于侦测事件，表示某路sensor
EventType	u8	事件类型 1：移动侦测 2：人形 3：声音	只针对侦测事件
FrameIndex	u32	帧索引	只针对视频
ClockTime	u64	运行时间戳，单位ms	clock_gettime(CLOCK_MONOTONIC, &fActive); ((unsigned long long)fActive.tv_sec * 1000000) + ((unsigned long long)fActive.tv_nsec / 1000000);
Timestamp	u64	UTC时间戳，单位ms	gettimeofday(&fTimestamp, NULL); ((unsigned long long)fTimestamp.tv_sec * 1000) + ((unsigned long long)fTimestamp.tv_usec / 1000);
FrameWidth	u16	帧宽度	这里在帧头中定义帧宽度，用于动态切换分辨率后，host端收到数据后进行有效过滤，比如不是当前要求分辨率，可以忽略。
FrameHeight	u16	帧高度	这里在帧头中定义帧高度，用于动态切换分辨率后，host端收到数据后进行有效过滤，比如不是当前要求分辨率，可以忽略。

VencType	u8	H264：1 H265：2 YUV：3 MJPEG：4	这里在帧头中定义编码类型，用于动态切换编码类型后，host端收到数据后进行有效过滤，比如不是当前要求的编码类型，可以忽略。
res[3]	u8	预留3字节，确保8字节对齐	

2.2帧数据定义

帧类型	类型ID	帧数据	备注
视频	1	一帧视频数据	I帧或P帧
侦测事件	2	一张jpg图片数据	
配网事件	3	配网信息	当使用通用控制命令（0x06）启动配网后，设备端会扫描二维码，有扫描结果就上报配网事件。当使用通用控制命令（0x06）停止配网后，设备端会停止扫描二维码，就不会再上报配网事件。
gps-rtk数据	4	一帧GPS-RTK数据	
IPC复位事件	5	只有帧头，没有数据体	
gsensor数据	6	一帧视频对应的gsensor数据	
音频	7	一帧音频数据	

2.2.1配网信息说明

字段名称	类型	定义	备注
wifi_ssid[32]	u8	Wifi SSID	
wifi_pwd[64]	u8	Wifi密码	
token[256]	u8	token字段	使用格式如下的： https://access_url/r/token

			access_url是平台访问地址，比如：activate.girafit.com。 token就是扫码获取到的token信息。 使用p2pDevice_BindDevice接口进行绑定，第一个参数传入https://access_url/r/token，第二个参数传入wifi_ssid。

2.2.2gsensor数据说明

UvcGyroData_t

字段名称	类型	定义	备注
frame_pts	u64	视频帧时间戳	
pts	u64	gsensor数据时间戳	
x	s32	方位x	
y	s32	方位y	
z	s32	方位z	
res[4]	u8	保留字段	8字节对齐

UvcGyroDataGroup_t

字段名称	类型	定义	备注
num	s32	UvcGyroData_t实际个数	
gsensor_id	s32	gsensor的ID，范围：0~3	
data[50]	UvcGyroData_t	一个视频帧对应的gsensor数据，最多包含50个	

3.使用说明(example)

3.1 HOST下发命令

3.1.1 控制接口定义

代码块

```
static void uvc_xu_ctrl(unsigned int unit, unsigned int selector,unsigned int query, unsigned int data_size, unsigned char* data_addr)
{
        int ret;
        struct uvc_xu_control_query x_ctrl = {0};

        x_ctrl.unit     = unit;
        x_ctrl.selector = selector;
        x_ctrl.query    = query;
        x_ctrl.size     = data_size;
        x_ctrl.data     = data_addr;

        ret = ioctl(g_fd, UVCIOC_CTRL_QUERY, &x_ctrl);
        if (ret < 0) {
                printf("UVCIOC_CTRL_QUERY fail\n");
        }
}
```

3.1.2 视频参数（命令ID：0x01）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID      0x0B    // 表示扩展命令
#define   XC_UVC_CMD_VIDEO_ATTR 0x01    //命令ID：表明控制项为视频参数
#define   UVC_SET_CUR           0x01    //query:设置当前值

//参数掩码设置
typedef enum
{
    VIDEO_ATTR_SWITCH_ON = 1 << 0,
    VIDEO_ATTR_RESOLUTION = 1 << 1,
    VIDEO_ATTR_VENC = 1 << 2,
    VIDEO_ATTR_OSD = 1 << 3,
    VIDEO_ATTR_TIME_LOCATION = 1 << 4,
    VIDEO_ATTR_LOGO_LOCATION = 1 << 5,
    VIDEO_ATTR_RATE_CTRL_MODE = 1 << 6,
    VIDEO_ATTR_OSD_REVERSE = 1 << 7,
    VIDEO_ATTR_GOP = 1 << 8,
    VIDEO_ATTR_FRAME_RATE = 1 << 9,
    VIDEO_ATTR_RATE = 1 << 10,
    VIDEO_ATTR_KEY_FRAME = 1 << 11,
    VIDEO_ATTR_HUMAN_BOX = 1 << 12,
    VIDEO_ATTR_QP = 1 << 13,
} VideoAttrCtrlBit;

```

```
typedef enum
{
        UVC_VENC_TYPE_H264 = 1,
        UVC_VENC_TYPE_H265,
        UVC_VENC_TYPE_YUV,
        UVC_VENC_TYPE_MJPEG,
} UvcVencType;

typedef struct
{
    unsigned int mask;
    unsigned char channel;
    unsigned char switch_on;
    unsigned short width;
    unsigned short height;
    unsigned char venc;
    unsigned char osd;
    unsigned int time_location;
    unsigned int logo_location;
    unsigned char rate_ctrl_mode;
    unsigned char osd_reverse;
    unsigned char GOP;
    unsigned char frame_rate;
    unsigned short rate;
    unsigned char key_frame;
    unsigned char human_box;
    unsigned char qp_type;      //1: cbr, 2: vbr
    unsigned char max_qp;       //帧最大qp，取值范围: [0,51]
    unsigned char min_qp;       //帧最小qp，取值范围: [0,max_qp]
    unsigned char max_i_qp;     //I帧的最大qp，[0,51]
    unsigned char min_i_qp;     //I帧的最小qp，[0,max_i_qp]
    unsigned char change_pos;   //数据类型xmedia_venc_param_h264_vbr结构体的change_pos，这个是 VBR 开始调整 QP 时的码率相对于最大码率的比例，
                                //对应需求应该是: 平均码率/最大码率 = change_pos，最大码率xmedia_venc_h264_vbr结构体的max_bit_rate，最大码率max_bit_rate，
                                //码率调整水线change_pos，尽可能码率维持: max_bit_rate * change_pos左右。注意change_pos范围50-100。
                                //只有VBR才会使用该参数，CBR不用该参数
    unsigned char res[26];
} UvcVideoAttr_t;

static void setVideoParam()
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_VIDEO_ATTR;
        unsigned int data_size = sizeof(UvcVideoAttr_t);
        unsigned char* data = malloc(data_size);
```

```
        UvcVideoAttr_t UvcVideoAttr = {0};

        if (data == NULL)
        {
            printf("Memory allocation failed\n");
            return;
        }

        memset(&UvcVideoAttr, 0, sizeof(UvcVideoAttr));
        UvcVideoAttr.mask =  VIDEO_ATTR_RATE_CTRL_MODE | VIDEO_ATTR_RATE | VIDEO_ATTR_VENC;//打开编码切换、时间水印位置功能
        UvcVideoAttr.channel = 0x0;
        UvcVideoAttr.switch_on = 0x0;
        UvcVideoAttr.GOP = 25;
        UvcVideoAttr.human_box = 0x1;
        UvcVideoAttr.osd = 0x0;
        UvcVideoAttr.frame_rate = 20;
        UvcVideoAttr.key_frame = 0x1;
        UvcVideoAttr.height = 720;
        UvcVideoAttr.width = 1280;
        UvcVideoAttr.logo_location = 0x77772222;
        UvcVideoAttr.osd_reverse = 0x01;
        UvcVideoAttr.time_location = 0x4444;
        UvcVideoAttr.rate_ctrl_mode = 0x1;
        UvcVideoAttr.rate = 800;
        UvcVideoAttr.venc = UVC_VENC_TYPE_H264;//264

        //qp值设置
        UvcVideoAttr.qp_type = 1;  //vbr
        UvcVideoAttr.max_qp = 51;
        UvcVideoAttr.min_qp = 24;
        UvcVideoAttr.max_i_qp = 51;
        UvcVideoAttr.min_i_qp = 20;
        UvcVideoAttr.change_pos = 90;

        memcpy(data, &UvcVideoAttr, sizeof(UvcVideoAttr));
        data_size = sizeof(UvcVideoAttr);

        //调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setVideoParam successful\n");

        free(data);
}
```

3.1.3 NTP时间（命令ID：0x02）

代码块

```
 #define   EXRNSION_UNIT_ID      0x0B    // 表示扩展命令
 #define   XC_UVC_CMD_NTP        0x02    //命令ID: 表明控制项为NTP
 #define   UVC_SET_CUR           0x01    //query:设置当前值

typedef struct
{
        unsigned int mask;
        int time_zone;
        unsigned int time_sec;
        unsigned int time_usec;
        unsigned char osd_time_format;
        unsigned char osd_date_format;
        unsigned char res[42];
} UvcNtp_t;

typedef enum
{
        NTP_TIMEZONE = 1 << 0,
        NTP_TIME = 1 << 1,
        OSD_TIME_FORMAT = 1 << 2,
        OSD_DATE_FORMAT = 1 << 3,
} NtpCtrlBit;

static void setNtpParam()
{
    unsigned int unit = EXRNSION_UNIT_ID;
    unsigned int selector = XC_UVC_CMD_NTP;
    unsigned int data_size = sizeof(UvcNtp_t);
    unsigned char* data = malloc(data_size);
    UvcNtp_t UvcNtpParam = {0};
    struct timeval tv;
    struct timezone tz;

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    //获取当前时间
    gettimeofday(&tv, &tz);

    UvcNtpParam.mask = NTP_TIMEZONE|NTP_TIME|OSD_TIME_FORMAT|OSD_DATE_FORMAT;
    UvcNtpParam.time_zone = 0;
    UvcNtpParam.time_sec = tv.tv_sec;    //使用系统当前时间戳
    UvcNtpParam.time_usec = tv.tv_usec;  //使用系统当前微秒数
```

```
    UvcNtpParam.osd_date_format = 1;     // yyyy/MM/dd格式
    UvcNtpParam.osd_time_format = 0;     //24h
    memcpy(data, &UvcNtpParam, data_size);

    // 调用控制接口
    uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
    printf("setNtpParam successful\n");

    free(data);
}
```

3.1.4 音频参数（命令ID：0x03）

代码块

```
 #define   EXRNSION_UNIT_ID      0x0B    // 表示扩展命令
 #define   XC_UVC_CMD_AUDIO_ATTR 0x03    //命令ID：表明控制项为音频参数
 #define   UVC_SET_CUR           0x01    //query:设置当前值

typedef struct
{
        unsigned int mask;
        unsigned char channel;
        unsigned char switch_on;
        unsigned char aenc;
        unsigned char bits_per_sample;
        unsigned short sample_rate;
        unsigned char volume;
        unsigned char mute;
        unsigned char res[48];
} UvcAudioAttr_t;

typedef enum
{
        AUDIO_ATTR_SWITCH_ON = 1 << 0,
        AUDIO_ATTR_AENC = 1 << 1,
        AUDIO_ATTR_BITS_PER_SAMPLE = 1 << 2,
        AUDIO_ATTR_SAMPLE_RATE = 1 << 3,
        AUDIO_ATTR_VOLUME = 1 << 4,
        AUDIO_ATTR_MUTE = 1 << 5,
} AudioAttrCtrlBit;

static void setAudioParam()
{
    unsigned int unit = EXRNSION_UNIT_ID;
    unsigned int selector = XC_UVC_CMD_AUDIO_ATTR;
```

```
    unsigned int data_size = sizeof(UvcAudioAttr_t);
    unsigned char* data = malloc(data_size); // 分配内存
    UvcAudioAttr_t UvcAudioParam = {0};

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

        UvcAudioParam.mask =AUDIO_ATTR_SWITCH_ON |AUDIO_ATTR_VOLUME|AUDIO_ATTR_MUTE ;
        UvcAudioParam.channel = 0x0;//麦克风
        UvcAudioParam.switch_on =0x1;
        UvcAudioParam.volume = 100;
        UvcAudioParam.mute = 0x0;

        memcpy(data,&UvcAudioParam, data_size);

    // 调用控制接口
       uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
       printf("setAudioParam successful.....\n");
       free(data);

}
```

3.1.5 侦测事件（命令ID：0x04）

代码块

```

//提前进行宏定义
 #define   EXRNSION_UNIT_ID         0x0B    // 表示扩展命令
 #define   XC_UVC_CMD_ALARM_DETECT  0x04    //命令ID: 表明控制项为侦测事件
 #define   UVC_SET_CUR              0x01    //query:设置当前值

 typedef enum
{
        ALARM_DETECT_REGION_POINTS = 1 << 0,
        ALARM_DETECT_MOTION_ON = 1 << 1,
        ALARM_DETECT_MOTION_SENSITIVITY = 1 << 2,
        ALARM_DETECT_HUMAN_ON = 1 << 3,
        ALARM_DETECT_HUMAN_SENSITIVITY = 1 << 4,
        ALARM_DETECT_FACE_ON = 1 << 5,
        ALARM_DETECT_FACE_SENSITIVITY = 1 << 6,
        ALARM_DETECT_MOTION_AUDIO_ALARM = 1 << 7,
        ALARM_DETECT_HUMAN_AUDIO_ALARM = 1 << 8,
```

```
18          ALARM_DETECT_FACE_AUDIO_ALARM = 1 << 9,
19          ALARM_DETECT_SOUND_ON = 1 << 10,
20          ALARM_DETECT_SOUND_SENSITIVITY = 1 << 11,
21  } AlarmDetectCtrlBit;
22
23  typedef struct
24  {
25          unsigned int mask;
26          unsigned int region_point[REGION_POINT_NUM];
27          unsigned char motion_on;
28          unsigned char motion_sensitivity;
29          unsigned char human_on;
30          unsigned char human_sensitivity;
31          unsigned char face_on;
32          unsigned char face_sensitivity;
33          unsigned char motion_audio_alarm;
34          unsigned char human_audio_alarm;
35          unsigned char face_audio_alarm;
36          unsigned char sound_det_on;
37          unsigned char sound_sensitivity;
38          unsigned char res[23];
39  } UvcAlarmDetect_t;
40
41  static void setDetectParam()
42  {
43      unsigned int unit = EXRNSION_UNIT_ID;
44      unsigned int selector = XC_UVC_CMD_ALARM_DETECT;
45      unsigned int data_size = sizeof(UvcAlarmDetect_t);
46      unsigned char* data = malloc(data_size);
47      UvcAlarmDetect_t UvcDetectParam = {0};
48
49      if (data == NULL) {
50          printf("Memory allocation failed\n");
51          return;
52      }
53
54          UvcDetectParam.mask =
    ALARM_DETECT_MOTION_AUDIO_ALARM|ALARM_DETECT_HUMAN_AUDIO_ALARM|ALARM_DETECT_FAC
    E_AUDIO_ALARM;//关闭声音告警
55
56          UvcDetectParam.region_point[0] = PACK_POINT(100, 100);
57          UvcDetectParam.region_point[1] = PACK_POINT(540, 100);
58          UvcDetectParam.region_point[2] = PACK_POINT(640, 240);
59          UvcDetectParam.region_point[3] = PACK_POINT(540, 380);
60          UvcDetectParam.region_point[4] = PACK_POINT(100, 380);
61          UvcDetectParam.region_point[5] = PACK_POINT(0, 240);
62
```

```
        UvcDetectParam.motion_on = 1;
        UvcDetectParam.motion_sensitivity = 70;
        UvcDetectParam.human_on = 1;
        UvcDetectParam.human_sensitivity = 50;
        UvcDetectParam.face_on = 1;
        UvcDetectParam.face_sensitivity = 60;
        UvcDetectParam.motion_audio_alarm = 0;
        UvcDetectParam.human_audio_alarm = 0;
        UvcDetectParam.face_audio_alarm = 0;
        UvcDetectParam.sound_det_on = 1;
        UvcDetectParam.sound_sensitivity = 60;

        memcpy(data, &UvcDetectParam, data_size);

        // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setDetectParam successful.....\n");
        free(data);
}
```

3.1.6 Sensor通道属性（命令ID：0x05）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID          0x0B    // 表示扩展命令
#define   XC_UVC_CMD_SENSOR_ATTR    0x05    //命令ID：表明控制项为设置sensor属性
#define   UVC_SET_CUR               0x01    //query:设置当前值

typedef struct
{
        unsigned int mask;
        unsigned char dev_id;
        unsigned char mirror;
        unsigned char flip;
        unsigned char res[53];
} UvcSensorAttr_t;

typedef enum
{
        SENSOR_ATTR_MIRROR = 1 << 0,
        SENSOR_ATTR_FLIP = 1 << 1,
} SensorAttrCtrlBit;

static void setSensorAttr()
{
```

```
    unsigned int unit = EXRNSION_UNIT_ID;
    unsigned int selector = XC_UVC_CMD_SENSOR_ATTR;
    unsigned int data_size = sizeof(UvcSensorAttr_t);
    unsigned char* data = malloc(data_size);
    UvcSensorAttr_t UvcSensorAttr = {0};

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
        UvcSensorAttr.mask = SENSOR_ATTR_MIRROR|SENSOR_ATTR_FLIP;//同时开启镜像和翻转
        UvcSensorAttr.dev_id = 0;
        UvcSensorAttr.mirror = 0x1;
        UvcSensorAttr.flip = 0x1;

        memcpy(data,&UvcSensorAttr, data_size);

        // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setSensorAttr successful\n");

        free(data);

}

```

3.1.7 通用控制命令（命令ID：0x06）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID          0x0B    // 表示扩展命令
#define   XC_UVC_CMD_COMMON_CTRL    0x06    //命令ID：表明控制项为通用控制
#define   UVC_SET_CUR               0x01    //query:设置当前值

typedef enum
{
        COMMON_CTRL_START_TALK = 1 << 0,
        COMMON_CTRL_START_BIND = 1 << 1,
        COMMON_CTRL_WHITE_LED = 1 << 2,
        COMMON_CTRL_ELE_ATTACK_1 = 1 << 3,
        COMMON_CTRL_ELE_ATTACK_1 = 1 << 4,
        COMMON_CTRL_ELE_LIGHT_1 = 1 << 5,
        COMMON_CTRL_ELE_LIGHT_2 = 1 << 6,
} CommonCtrlBit;
```

```c

typedef struct
{
        unsigned int mask;
        unsigned char start_talk;
        unsigned char start_bind;
        unsigned char white_led_on;
        unsigned char ele_attack1;
        unsigned char ele_attack2;
        unsigned char ele_light1;
        unsigned char ele_light2;
        unsigned char res[49];
} UvcCommonCtrl_t;


static void setCommonCtrlParam()
{
    unsigned int unit = EXRNSION_UNIT_ID;
    unsigned int selector = XC_UVC_CMD_COMMON_CTRL;
    unsigned int data_size = sizeof(UvcCommonCtrl_t);
    unsigned char* data = malloc(data_size);
    UvcCommonCtrl_t UvcCommonCtrlParam = {0};

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
        UvcCommonCtrlParam.mask = COMMON_CTRL_START_BIND|COMMON_CTRL_START_TALK|COMMON_CTRL_WHITE_LED;
        UvcCommonCtrlParam.start_talk = 0; //关闭对讲
        UvcCommonCtrlParam.start_bind = 1;//启动配网
        UvcCommonCtrlParam.white_led_on = 1;//开启白光灯
        UvcCommonCtrlParam.ele_attack1 = 0;//电子枪1攻击
        UvcCommonCtrlParam.ele_attack2 = 0;//电子枪2攻击
        UvcCommonCtrlParam.ele_light1 = 0;//电子枪1照明
        UvcCommonCtrlParam.ele_light2 =0;//电子枪2照明

        memcpy(data,&UvcCommonCtrlParam, data_size);

        // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setCommonCtrlParam successful\n");

        free(data);
}

```

3.1.8 获取设备信息（命令ID：0x07）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID          0x0B    // 表示扩展命令
#define   XC_UVC_CMD_DEVICE_INFO    0x07    //命令ID: 表明控制项为获取设备信息
#define   UVC_GET_CUR               0x81    //query:获取当前值

typedef struct
{
        unsigned char version[32];
        unsigned char res[28];
}UvcDeviceInfo_t;

static void getDeviceInfo(void)
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_DEVICE_INFO;
        unsigned int data_size = sizeof(UvcDeviceInfo_t);
        unsigned char* data = malloc(data_size);
        UvcDeviceInfo_t UvcDeviceInfo = {0};

        if (data == NULL) {
            printf("Memory allocation failed\n");
            return;
        }

         // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_GET_CUR, data_size, data);
        printf("getDeviceInfo successful\n");
        memcpy(&UvcDeviceInfo,data,data_size);

        printf("UvcDeviceInfo.version : %s\n",UvcDeviceInfo.version);
        free(data);
}

```

3.1.9 CPE控制（命令ID：0x08）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID          0x0B    // 表示扩展命令
#define   XC_UVC_CMD_CPE_CTRL       0x08    //命令ID: 表明控制项为CPE控制
#define   UVC_SET_CUR               0x01    //query:设置当前值
```

```c

typedef struct
{
        unsigned int mask;
        unsigned char cpe_mode;
        unsigned char set_ssid;
        unsigned char encryption;
        unsigned char password_len;
        unsigned char ssid[32];
        unsigned char password_tail[4];
        unsigned char res[16];
} UvcCpeCtrl_t;

typedef enum
{
        CPE_CTRL_SET_CPE_MODE = 1 << 0,
} CpeCtrlBit;

typedef enum
{
        UVC_CPE_MODE_AUTO = 0,
        UVC_CPE_MODE_4G,
        UVC_CPE_MODE_WIFI,
        UVC_CPE_MODE_NUM,
} UvcCpeMode;

static void setCpeParam()
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_CPE_CTRL;
        unsigned int data_size = sizeof(UvcCpeCtrl_t);
        unsigned char* data = malloc(data_size);
        UvcCpeCtrl_t UvcCpeParam = {0};

        UvcCpeParam.mask = CPE_CTRL_SET_CPE_MODE;
        UvcCpeParam.cpe_mode = UVC_CPE_MODE_WIFI;
        UvcCpeParam.set_ssid = 1;//下发新的ssid
        UvcCpeParam.encryption = 1;//wifi加密
        UvcCpeParam.password_len = 10;
        strncpy((char*)UvcCpeParam.ssid, "XY_test", sizeof(UvcCpeParam.ssid) - 1);

        memcpy(data,&UvcCpeParam, data_size);
        //调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setCpeParam successful\n");

```

```
        free(data);

}
```

3.1.10 下发WIFI密码（命令ID：0x09）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID          0x0B    // 表示扩展命令
#define   XC_UVC_CMD_WIFI_PASSWORD  0x09    //命令ID: 表明控制项为设置wiifi密码
#define   UVC_SET_CUR               0x01    //query:设置当前值

typedef struct
{
      unsigned char password[60];
} UvcWifiPassword_t;

static void setWifiPassword()
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_WIFI_PASSWORD;
        unsigned int data_size = sizeof(UvcWifiPassword_t);
        unsigned char* data = malloc(data_size);
        UvcWifiPassword_t UvcWifiPassword = {0};

        strcpy(UvcWifiPassword.password,"Share.123");
        memcpy(data,&UvcWifiPassword, data_size);
        //调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setWifiPassword successful\n");

        free(data);
}
```

3.1.11 获取CPE信息（命令ID：0x0A）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID          0x0B    // 表示扩展命令
```

```
#define   XC_UVC_CMD_GET_CPE_INFO   0x0A    //命令ID：表明控制项为CPE控制
#define   UVC_GET_CUR               0x81    //query:获取当前值

typedef struct
{
        int percent;
        int dbm;
        unsigned char cpe_mode;
        unsigned char res[51];
} UvcCpeInfo_t;

static void getCpeParam()
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_GET_CPE_INFO;
        unsigned int data_size = sizeof(UvcCpeInfo_t);
        unsigned char* data = malloc(data_size);
        UvcCpeInfo_t UvcCpeInfo = {0};

         // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_GET_CUR, data_size, data);
        printf("getCpeParam successful\n");
        memcpy(&UvcCpeInfo,data,data_size);

        printf("UvcCpeInfo.percent : %d\n",UvcCpeInfo.percent);
        printf("UvcCpeInfo.dbm : %d\n",UvcCpeInfo.dbm);
        printf("UvcCpeInfo.cpe_mode : %d\n",UvcCpeInfo.cpe_mode);

        free(data);

}
```

3.1.12 host端状态通知（命令ID：0x0B）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID              0x0B    // 表示扩展命令
#define   XC_UVC_CMD_HOST_STATUS_NOTIFY 0X0B    //命令ID：表明控制项为下发host端状态
#define   UVC_SET_CUR                   0x01    //query:设置当前值

typedef enum
{
      STATUS_CTRL_HOST_STATUS = 1 << 0,
} HostStatusCtrlBit;
```

```c

typedef enum
{
        HOST_STATUS_PREPARE_BINDING = 0,
        HOST_STATUS_CONNECT_INTERNET,
        HOST_STATUS_CONNECT_CPE,
        HOST_STATUS_CONNECT_CLOUD,
        HOST_STATUS_OTHER_NET_ERROR,
        HOST_STATUS_BIND_ACCOUNT,
        HOST_STATUS_DISCONNECT_CLOUD,
        HOST_STATUS_RECONNECT_CLOUD,
        HOST_STATUS_NUM,
} HOST_STATUS_E;

typedef struct
{
        unsigned int mask;
        unsigned int host_status;
        unsigned char res[52];
}UvcHostStatus_t;


static void setHostStatusNotify()
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_HOST_STATUS_NOTIFY;
        unsigned int data_size = sizeof(UvcHostStatus_t);
        unsigned char* data = malloc(data_size);
        UvcHostStatus_t UvcHostStatus = {0};

        UvcHostStatus.mask = STATUS_CTRL_HOST_STATUS ;
        UvcHostStatus.host_status = HOST_STATUS_CONNECT_INTERNET;//连接internet中

        memcpy(data,&UvcHostStatus,data_size);
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setHostStatusNotifysuccessful\n");

        free(data);

}
```

3.1.13 OTA升级（命令ID：0x0C）

代码块

```c
//提前进行宏定义
#define   EXRNSION_UNIT_ID              0x0B    // 表示扩展命令
#define   XC_UVC_CMD_OTA                0x0C    //命令ID: 表明控制项为下发OTA升级
#define   UVC_SET_CUR                   0x01    //query:设置当前值
#define   UVC_GET_CUR                   0x81    //query:获取当前值

typedef struct
{
        unsigned int length;
        unsigned char md5[32];
        unsigned char start;
        unsigned char ota_type;
        unsigned char ota_status;
        unsigned char start_result;
        unsigned char res[20];
} UvcOta_t;

static void setOtaUpgrade()
{

        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_OTA;
        unsigned int data_size = sizeof(UvcOta_t);
        unsigned char* data = malloc(data_size);
        UvcOta_t UvcOtaUpgrade = {0};

        UvcOtaUpgrade.length = getLengthFromFile(OTA_FILE_PATH);
        char *md5 = calculateMd5(OTA_FILE_PATH);
        strcpy(UvcOtaUpgrade.md5,md5);
        UvcOtaUpgrade.ota_type = 1;//OTA类型: 1: IPC固件升级   2: CPE固件升级
        UvcOtaUpgrade.start = 1;  //1: 启动传输   0: 结束传输

        printf("UvcOtaUpgrade.length:%d\n",UvcOtaUpgrade.length);
        printf("UvcOtaUpgrade.md5:%s\n",UvcOtaUpgrade.md5);
        printf("UvcOtaUpgrade.ota_type:%d\n",UvcOtaUpgrade.ota_type);
        printf("UvcOtaUpgrade.start:%d\n",UvcOtaUpgrade.start);

        memcpy(data,&UvcOtaUpgrade,data_size);
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
        printf("setOtaUpgrade successful\n");

        free(data);
        free(md5);

}

static void getOtaStatus()
```

```
48  {
49
50          unsigned int unit = EXRNSION_UNIT_ID;
51          unsigned int selector = XC_UVC_CMD_OTA;
52          unsigned int data_size = sizeof(UvcOta_t);
53          unsigned char* data = malloc(data_size);
54          UvcOta_t UvcOtaStatu = {0};
55
56          // 调用控制接口
57          uvc_xu_ctrl(unit, selector, UVC_GET_CUR, data_size, data);
58          printf("getOtaStatus successful\n");
59          memcpy(&UvcOtaStatu,data,data_size);
60
61          printf("UvcOtaStatu.ota_status:%d\n",UvcOtaStatu.ota_status);
62          printf("UvcOtaStatu.start_result:%d\n",UvcOtaStatu.start_result);
63
64          free(data);
65
66  }
67
```

3.1.14 获取SD卡信息（命令ID：0x0D）

代码块

```
1   //提前进行宏定义
2   #define   EXRNSION_UNIT_ID              0x0B    // 表示扩展命令
3   #define   XC_UVC_CMD_GET_SDCARD_INFO    0X0D    //命令ID: 获取SD卡信息
4   #define   UVC_GET_CUR                   0x81    //query:获取当前值
5
6   typedef struct
7   {
8           int total_size;
9           int used_size;
10          unsigned char status;
11          unsigned char volume[11];
12          unsigned char res[40];
13  } UvcSDCardInfo_t;
14
15  static int getSdcardInfo()
16  {
17
18          unsigned int unit = EXRNSION_UNIT_ID;
19          unsigned int selector = XC_UVC_CMD_GET_SDCARD_INFO;
20          unsigned int data_size = sizeof(UvcSDCardInfo_t);
21          unsigned char* data = malloc(data_size);
```

```
        UvcSDCardInfo_t UvcSdcardInfo = {0};

         // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_GET_CUR, data_size, data);
        printf("getSdcardInfo successful\n");
        memcpy(&UvcSdcardInfo,data,data_size);

        printf("UvcSdcardInfo.total_size : %d\n",UvcSdcardInfo.total_size);
        printf("UvcSdcardInfo.used_size : %d\n",UvcSdcardInfo.used_size);
        printf("UvcSdcardInfo.status : %hhu\n",UvcSdcardInfo.status);
        printf("UvcSdcardInfo.volume:%s\n",UvcSdcardInfo.volume);

        free(data);

}

```

3.1.15 获取网络接口信息（命令ID：0x0E）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID              0x0B    // 表示扩展命令
#define   XC_UVC_CMD_GET_ETH_INFO       0X0E    //命令ID：获取网络接口信息
#define   UVC_GET_CUR                   0x81    //query:获取当前值

typedef struct
{
        unsigned char duplex;
        unsigned char operstate;
        signed short speed;
        unsigned char res[56];
} UvcEthInfo_t;

static int getNetworkInfo()
{

        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_GET_ETH_INFO;
        unsigned int data_size = sizeof(UvcEthInfo_t);
        unsigned char* data = malloc(data_size);
        UvcEthInfo_tUvcNetworkInfo = {0};

         // 调用控制接口
```

```
        uvc_xu_ctrl(unit, selector, UVC_GET_CUR, data_size, data);
        printf("getNetworkInfo successful\n");
        memcpy(&UvcNetworkInfo,data,data_size);

        printf("UvcNetworkInfo.duplex : %hhu\n",UvcNetworkInfo.duplex);
        printf("UvcNetworkInfo.operstate : %hhu\n",UvcNetworkInfo.operstate);
        printf("UvcNetworkInfo.speed : %hd\n",UvcNetworkInfo.speed);

        free(data);

}

```

3.1.16 设置host端信息（命令ID：0x0F）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID              0x0B    // 表示扩展命令
#define   XC_UVC_CMD_SET_HOST_INFO      0x0F    //设置host端信息
#define   UVC_SET_CUR                   0x01    //query:设置当前值

typedef struct
{
        unsigned int mask;
        unsigned char MAC[18];
        unsigned char ref[38];

}UvcHostInfo_t;

static int setHostInfo()
{

        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_SET_HOST_INFO;
        unsigned int data_size = sizeof(UvcHostInfo_t);
        unsigned char* data = malloc(data_size);
        UvcHostInfo_t UvcHostInfo = {0};

        UvcHostInfo.mask = 1 << 0;
        strcpy(UvcHostInfo.MAC,"F2:7F:41:4B:DE:DC");

        // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_SET_CUR, data_size, data);
```

```
        printf("setHostInfo successful\n");

        free(data);

}

```

3.1.17 获取CPE固件版本（命令ID：0x10）

代码块

```
//提前进行宏定义
#define   EXRNSION_UNIT_ID              0x0B    // 表示扩展命令
#define   XC_UVC_CMD_GET_CPE_VERSION    0x10    //设置host端信息
#define   UVC_GET_CUR                   0x81    //query:获取当前值

typedef struct
{
        unsigned char version[32];
        unsigned char res[28];
}UvcCpeVersion_t;

static int getCpeVersion()
{
        unsigned int unit = EXRNSION_UNIT_ID;
        unsigned int selector = XC_UVC_CMD_GET_CPE_VERSION;
        unsigned int data_size = sizeof(UvcCpeVersion_t);
        unsigned char* data = malloc(data_size);
        UvcCpeVersion_t UvcCpeVersion = {0};

         // 调用控制接口
        uvc_xu_ctrl(unit, selector, UVC_GET_CUR, data_size, data);
        printf("getCpeVersion successful\n");
        memcpy(&UvcCpeVersion,data,data_size);

        printf("UvcCpeVersion.version : %s\n",UvcCpeVersion.version);

        free(data);

}
```

3.2 device向host发送消息

3.2.1 侦测事件获取

已知帧头标志为：

代码块

```
#define MAKEFLAG(ch0, ch1, ch2, ch3)                                \
                ((unsigned int)(char)(ch0) | ((unsigned int)(char)(ch1) << 8) |   \
                ((unsigned int)(char)(ch2) << 16) | ((unsigned int)(char)(ch3) << 24 ))

#define UVC_FRAME_HEADER_FLAG        MAKEFLAG('D','Y','6','6')  //0x36365944
```

获取帧头信息：

代码块

```
#define MAKEFLAG(ch0, ch1, ch2, ch3)                                \
                ((unsigned int)(char)(ch0) | ((unsigned int)(char)(ch1) << 8) |   \
                ((unsigned int)(char)(ch2) << 16) | ((unsigned int)(char)(ch3) << 24 ))

#define UVC_FRAME_HEADER_FLAG        MAKEFLAG('D','Y','6','6')  //0x36365944

typedef enum
{
        UVC_FRAME_TYPE_VIDEO = 1,
        UVC_FRAME_TYPE_DETECT,
        UVC_FRAME_TYPE_NETWORK,
        UVC_FRAME_TYPE_GPS_RTK,
        UVC_FRAME_TYPE_IPC_RESET,
        UVC_FRAME_TYPE_GENSOR_DATA
} UvcFrameType;

typedef enum
{
        UVC_EVENT_TYPE_MOTION = 1,
        UVC_EVENT_TYPE_HUMAN,
        UVC_EVENT_TYPE_SOUND,
        UVC_EVENT_TYPE_MAX,
} UvcEventType;

typedef struct
{
    unsigned int HeaderFlag;
    unsigned int DataLen;
    unsigned char FrameType;
```

```
    unsigned char KeyFrame;
    unsigned char Channel;
    unsigned char EventType;
    unsigned int FrameIndex;
    unsigned long long ClockTime;
    unsigned long long Timestamp;
    unsigned short FrameWidth;
    unsigned short FrameHeight;
    unsigned char VencType;
    unsigned char res[3];
} UvcFrameHeader_t;

typedef struct
{
    unsigned long long frame_pts;
    unsigned long long pts;
    int x;
    int y;
    int z;
    unsigned char res[4];
} UvcGyroData_t;

typedef struct
{
    int num;
    int gsensor_id;
    UvcGyroData_t data[UVC_GSENSOR_DATA_NUM];
} UvcGyroDataGroup_t;

typedef struct
{
        unsigned char ssid[UVC_WIFI_SSID_SIZE];
        unsigned char pwd[UVC_PASSWORD_SIZE];
        unsigned char token[UVC_URL_SIZE];
} UvcBindInfo_t;

static unsigned int g_all_stream_len = 0;
static void check_stream(unsigned char *stream, unsigned int length)
{
        UvcFrameHeader_t vl_header = {0};
        unsigned int cur_pos = 0;
        UvcBindInfo_t bind_info = {0};
        char gps_str[2048] = {0};
        UvcGyroDataGroup_t gSensor_data = {0};

        while(cur_pos < length)
        {
```

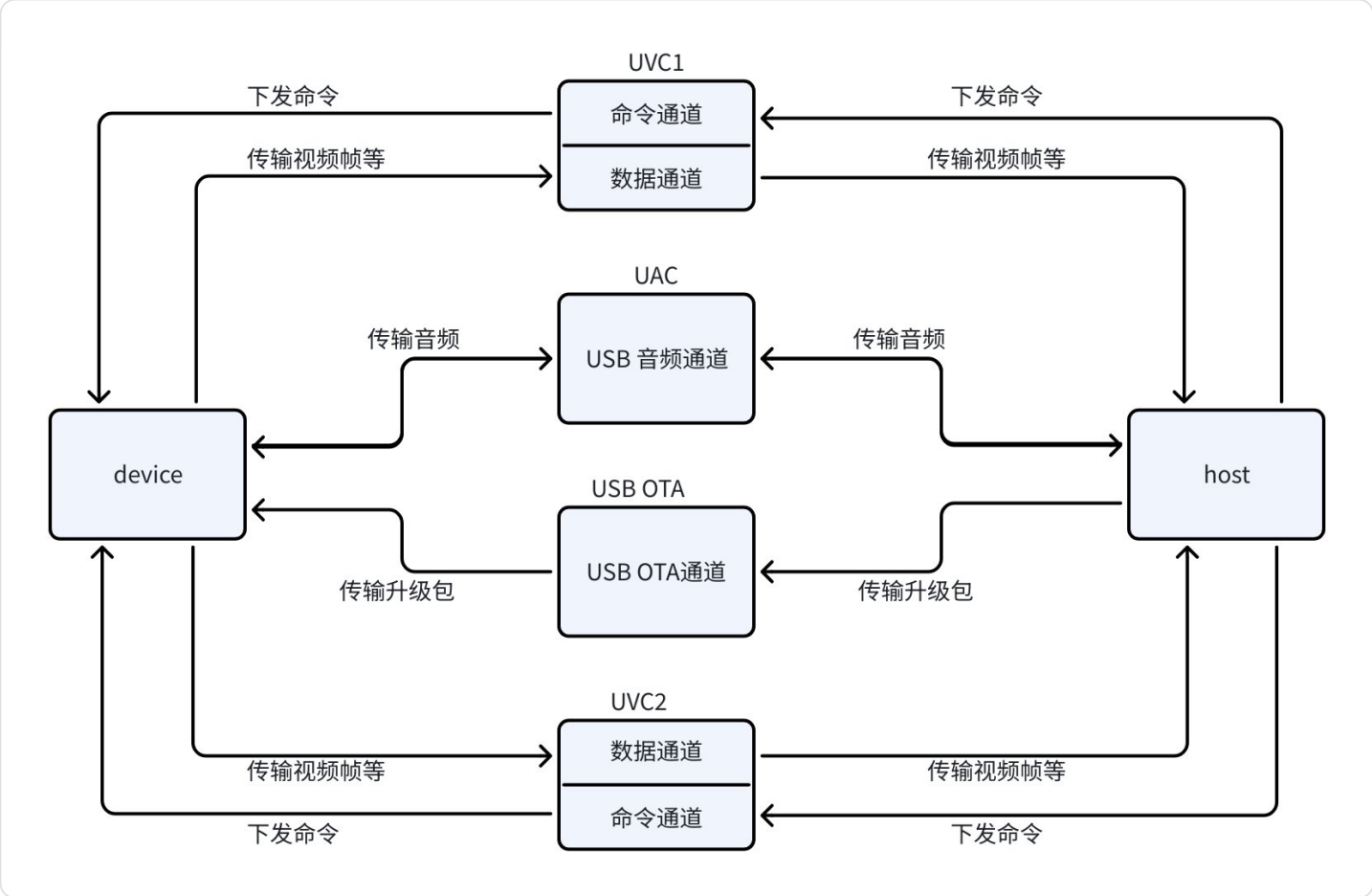
```
            if((stream[cur_pos] == 0x02 && stream[cur_pos+1] == 0x80) ||
               (stream[cur_pos] == 0x02 && stream[cur_pos+1] == 0x81) ||
               (stream[cur_pos] == 0x02 && stream[cur_pos+1] == 0x82) ||
               (stream[cur_pos] == 0x02 && stream[cur_pos+1] == 0x83))
            {
                cur_pos += 2;
            }

            if(cur_pos + sizeof(UvcFrameHeader_t) > length)
            {
                    break;
            }
            if(0x44 == stream[cur_pos] && 0x59 == stream[cur_pos+1] && 0x36 == stream[cur_pos+2] && 0x36 == stream[cur_pos+3])
            {
                    memset(&vl_header, 0, sizeof(vl_header));
                    memcpy(&vl_header, stream+cur_pos, sizeof(vl_header));
            }
            else
            {
                    cur_pos += 1;
                    continue;
            }

            if(vl_header.Channel == 0){
                  //printf("DataLen:%d FrameType:%d Channel:%d EventType:%d FrameIndex:%d FrameWidth:%d FrameHeight:%d ClockTime:%llums Timestamp:%llums VencType:%d keyframe:%d\n",vl_header.DataLen,vl_header.FrameType, vl_header.Channel, vl_header.EventType, vl_header.FrameIndex,vl_header.FrameWidth, vl_header.FrameHeight,vl_header.ClockTime,vl_header.Timestamp,vl_header.VencType,vl_header.KeyFrame);
            }

            if(UVC_FRAME_TYPE_VIDEO == vl_header.FrameType)
            {
                    //printf("VHeader.FrameType is UVC_FRAME_TYPE_VIDEO\n");
            }

            else if(UVC_FRAME_TYPE_DETECT == vl_header.FrameType)
            {
                printf("FrameType:%d Channel:%d EventType:%d FrameIndex:%d FrameWidth:%d FrameHeight:%d ClockTime:%llums Timestamp:%llums VencType:%d\n",vl_header.FrameType, vl_header.Channel, vl_header.EventType, vl_header.FrameIndex,vl_header.FrameWidth, vl_header.FrameHeight,vl_header.ClockTime,vl_header.Timestamp,vl_header.VencType);
```

```

                char snap_file[128] = {0};
                if(UVC_EVENT_TYPE_MOTION == vl_header.EventType)
                {
                        snprintf(snap_file, sizeof(snap_file)-1, "/mnt/sdcard/test_chan%d_motion.jpg", vl_header.Channel);
                }
                else(UVC_EVENT_TYPE_HUMAN == vl_header.EventType)
                {
                        snprintf(snap_file, sizeof(snap_file)-1, "/mnt/sdcard/test_chan%d_human.jpg", vl_header.Channel);
                }
                FILE* fp = fopen(snap_file, "w+");
                if(NULL != fp)
                {
                        chmod(snap_file, S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH);
                        int ret = fwrite(stream+cur_pos+sizeof(UvcFrameHeader_t), 1, vl_header.DataLen - sizeof(UvcFrameHeader_t), fp);
                        if (ret != (vl_header.DataLen - sizeof(UvcFrameHeader_t)))
                        {
                                printf("fwrite fail! ret=%d, JpgDataLen=%d\n", ret, (vl_header.DataLen - sizeof(UvcFrameHeader_t)));
                        }
                        fflush(fp);
                        fclose(fp);
                        fp = NULL;
                }
                if(UVC_EVENT_TYPE_SOUND == vl_header.EventType){
                        printf("detect sound.........\n");
                }

            }
            else if(UVC_FRAME_TYPE_NETWORK == vl_header.FrameType)
            {
                memcpy(&bind_info, stream+cur_pos+sizeof(UvcFrameHeader_t), sizeof(bind_info));
                printf("recv bind info!!! ssid:%s pwd:%s token:%s\n", bind_info.ssid, bind_info.pwd, bind_info.token);
            }
            else if(UVC_FRAME_TYPE_GPS_RTK == vl_header.FrameType)
            {
                memset(gps_str, 0, sizeof(gps_str));
                memcpy(gps_str, stream+cur_pos+sizeof(UvcFrameHeader_t), vl_header.DataLen - sizeof(UvcFrameHeader_t));
                printf("recv gps: %s\n", gps_str);
```

```
            }
            else if(UVC_FRAME_TYPE_IPC_RESET == vl_header.FrameType)
            {
                printf("VHeader.FrameType is UVC_FRAME_TYPE_IPC_RESET\n");
            }
            else if(UVC_FRAME_TYPE_GENSOR_DATA == vl_header.FrameType)
            {
                memset(&gSensor_data,0,sizeof(gSensor_data));
                memcpy(&gSensor_data,stream+cur_pos+sizeof(UvcFrameHeader_t), sizeof(gSensor_data));
                //printf("VHeader.FrameType is UVC_FRAME_TYPE_GENSOR_DATA\n");
                for(int i=0;i<gSensor_data.num;i++)
                {
                        //printf("gSensor_data.num:%d gSensorID:%d frame_pts:%llu pts:%llu x:%d y:%d z:%d\n",1+i,gSensor_data.gsensor_id,gSensor_data.data[i].frame_pts,gSensor_data.data[i].pts,gSensor_data.data[i].x,gSensor_data.data[i].y,gSensor_data.data[i].z);
                }
            }
            else
            {
                    printf("VHeader.FrameType is error!!!\n");
            }
            cur_pos += vl_header.DataLen;
        }
        g_all_stream_len += length;
}
```

4.UVC传输框架集成

4.1通信框图说明



4.2通道说明

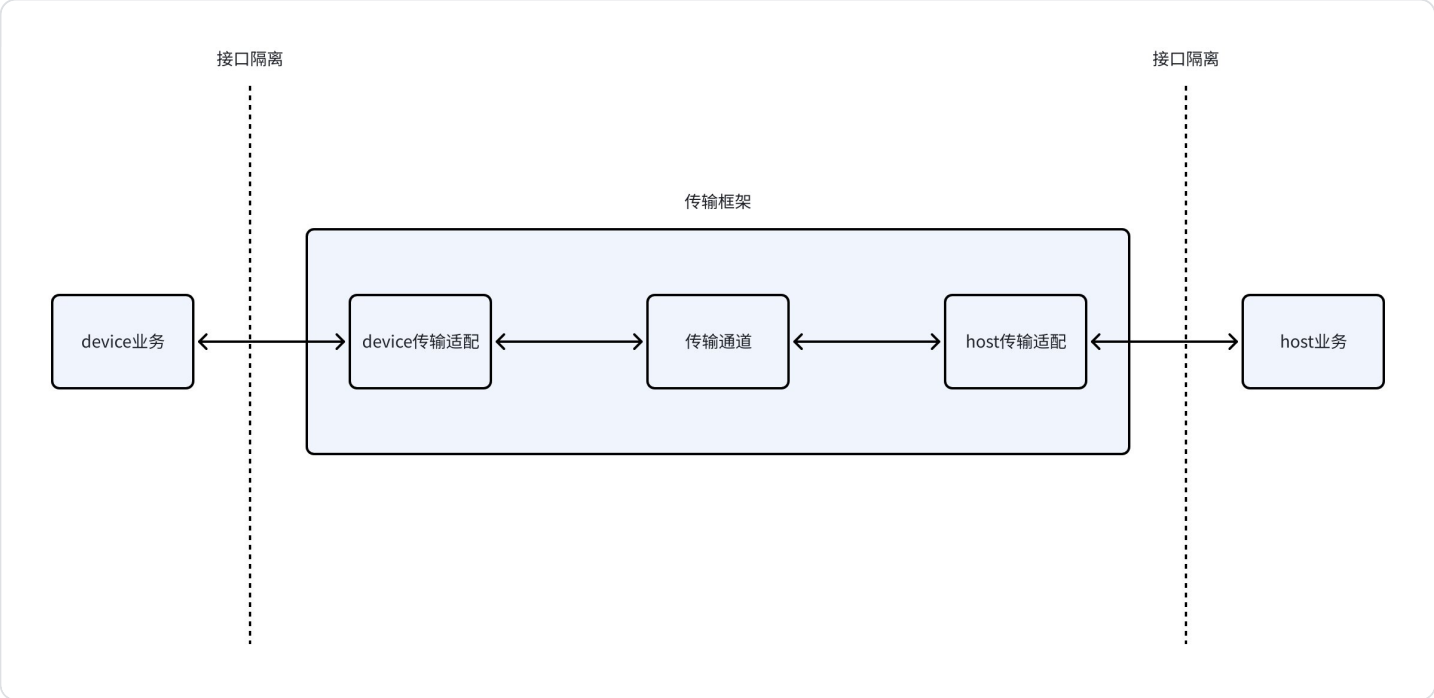
UVC1：数据通道，用于传输视频帧、侦测事件、扫码绑定信息、恢复出厂事件、GPS数据、GSensor数据，由device向host传输。命令通道，由host向device传输命令。

UVC2：同UVC1，功能一样。

UAC：用于传输音频，支持host和device之间双向传输。

USB OTA：用于传输OTA升级包，由host向device传输。

4.3框架设计建议



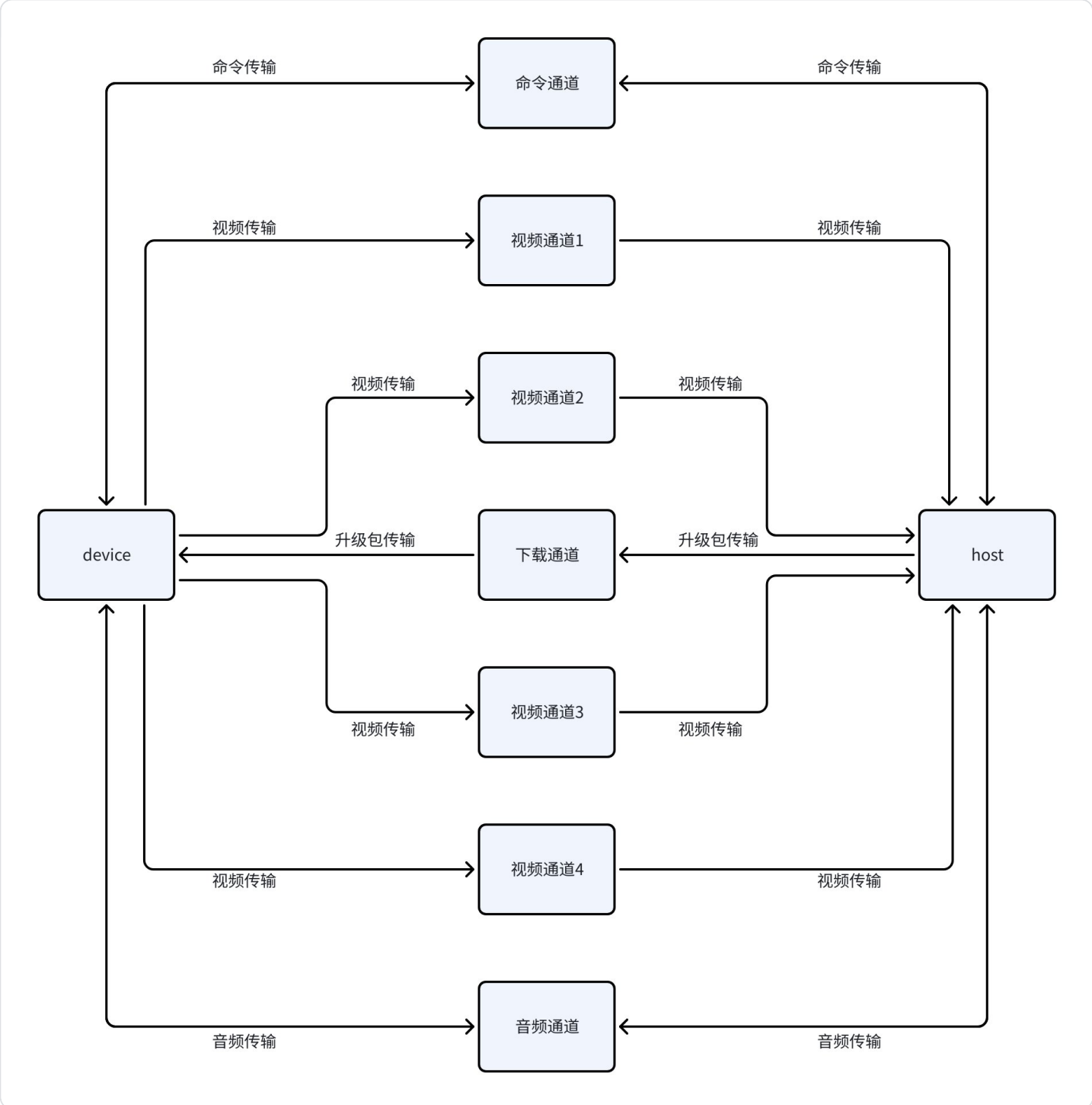
在框架设计的时候，注意解耦处理：

1）增加device业务和传输框架之间解耦，可以通过接口隔离。当传输框架更换之后，device端业务代码是可复用的。

2）对于host端也是同样的道理，在host端业务和传输框架之间解耦，可以通过接口隔离。当传输框架更换之后，host端业务代码是可复用的。

5.P2P传输框架集成

5.1通信框图说明



5.2P2P通道说明

```
#define P2PKIT_CMD_CHN          0       /*命令通道*/
#define P2PKIT_SPEAK_CHN        1       /*对讲通道*/
#define P2PKIT_LIVE_CHN         2       /*实时流*/
#define P2PKIT_DOWNLOAD_CHN  4          /*下载*/
#define P2PKIT_DATA_CHN_5    5          /*reserve*/
#define P2PKIT_DATA_CHN_6    6          /*reserve*/
#define P2PKIT_DATA_CHN_7    7          /*reserve*/
```

命令通道：P2PKIT_CMD_CHN，支持host和device之间双向传输命令，支持device向host端传输侦测事件、扫码绑定信息、恢复出厂事件、GPS数据、GSensor数据。

视频通道1：P2PKIT_LIVE_CHN，由device向host端发送sensor0所有编码通道数据。

视频通道2：P2PKIT_DATA_CHN_5，由device向host端发送sensor1所有编码通道数据。

视频通道3：P2PKIT_DATA_CHN_6，由device向host端发送sensor2所有编码通道数据。

视频通道4：P2PKIT_DATA_CHN_7，由device向host端发送sensor3所有编码通道数据。

下载通道：P2PKIT_DOWNLOAD_CHN，由host向device发送OTA升级包。

音频通道：P2PKIT_SPEAK_CHN，支持host和device之间双向传输音频。

5.3基于P2P传输UVC命令

UVC命令是1.2中定义的命令，最开始就是为UVC传输设计的，后续切换P2P传输兼容原有的UVC命令。

```
#define UVC_CMD_HEAD_FLAG          0x78789999
#define XC_UVC_CMD_DATA_SIZE (60)
```

uvc_control_query是P2P新定义的数据结构，尽量与1.1中uvc_xu_control_query保持一致。

```
typedef struct
{
    unsigned int flag;
    unsigned int unit;
    unsigned char selector;
    unsigned char query;
    unsigned short length;
    unsigned char data[XC_UVC_CMD_DATA_SIZE];
} uvc_control_query;
```

uvc_control_query新增加flag标记，UVC_CMD_HEAD_FLAG作为1.2中命令的标识。

当传输UVC命令

1）把uvc_control_query中flag设置为UVC_CMD_HEAD_FLAG。

2）unit要设置为1.1中定义的扩展单元EXRNSION_UNIT_ID。

3）selector还是1.2中定义的命令。

4）query是1.1中UVC_SET_CUR、UVC_GET_CUR之类定义。

5）length默认为XC_UVC_CMD_DATA_SIZE。

6）data为1.2中命令定义的数据结构。

5.4基于P2P传输P2P命令

#define P2PKIT_CMD_HEAD_FLAG 0x76768888

#define XC_P2P_UNIT_ID (0x1000)

#define XC_P2P_CMD_STREAM_ON (0x01)

#define XC_P2P_CMD_STREAM_OFF (0x02)

#define XC_P2P_CMD_LOGIN (0x03)

上面定义的XC_P2P_CMD_开头的命令，是针对P2P流程新增的命令。

P2PKIT_CMD_HEAD_FLAG是P2P流程新增命令的标识，赋值uvc_control_query中flag字段。

当传输P2P命令

1）把uvc_control_query中flag设置为P2PKIT_CMD_HEAD_FLAG。

2）unit设置为XC_P2P_UNIT_ID。

3）selector是P2P命令。

4）query是1.1中UVC_SET_CUR、UVC_GET_CUR之类定义。

5）length默认为XC_UVC_CMD_DATA_SIZE。

6）data为P2P命令定义的数据结构。

6.关键定义

6.1IPC对应UVC的PID和VID定义

#define VID 0x3337

#define PID 0x4321

6.2拓展板对应UVC的PID和VID定义

#define VID 0x3338

#define PID 0x4322